Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 3

За седьмой семестр

По дисциплине «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:
Студент 4 курса
Группы ИИ-26
Черноиван А.Н.
Проверил:
Хацкевич А.С.

2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Инструкция по подготовке стенда на личных устройствах: Так как программы eBPF компилируются «на лету» под конкретное ядро ОС, работа выполняется **исключительно в среде Linux**.

- **Для пользователей Windows:** необходимо использовать подсистему WSL2. Откройте PowerShell от имени администратора и выполните: wsl --install -d Ubuntu-22.04. После перезагрузки работайте внутри терминала Ubuntu.
- **Для пользователей macOS (Apple Silicon M1/M2/M3 или Intel):** необходимо установить полноценную виртуальную машину с Ubuntu 22.04 (рекомендуется использовать бесплатный гипервизор UTM или VirtualBox).
- **Внимание!** Использование Docker Desktop на Windows и macOS для данной работы **запрещено**, так как встроенная легковесная ВМ Docker не содержит заголовочных файлов ядра, необходимых для компиляции eBPF-программ.

Настройка окружения внутри Ubuntu/WSL2:

1. Обновите пакеты базы данных: sudo apt-get update
2. Установите базовые утилиты BCC и Python-библиотеки: sudo apt-get install -y bpfcc-tools python3-bpfcc
3. Установите заголовочные файлы ядра:
    - **Для полноценных виртуальных машин (VirtualBox/UTM):** выполните sudo apt-get install -y linux-headers-$(uname -r)
    - **Для пользователей WSL2 (Windows):** шаг с установкой linux-headers пропускается, так как Microsoft использует кастомное монолитное ядро. Установленных пакетов из пункта 2 будет достаточно для работы базовых kprobes. Если BCC всё же потребует исходники ядра при запуске скрипта, вам придется перейти на использование полноценной виртуальной машины (VirtualBox).
4. Все скрипты, взаимодействующие с подсистемой eBPF, должны запускаться строго с привилегиями суперпользователя (sudo python3 script.py).

Индивидуальный вариант

Вариант	**Целевой системный вызов (или tracepoint)**	**Условие аномального поведения (Фильтр ядра)**
7	sys_enter_mkdirat	Попытка создания скрытой директории (имя начинается с точки .)

Для каждого из модулей (заданий) в отчете необходимо привести:

Задание 1 (Базовый перехват запуска процессов):

1. Листинг eBPF-программы (C-код для ядра и Python-обертка) для перехвата вызова execve.

```
#!/usr/bin/python3
# Задание 1. Базовый перехват запуска процессов (execve)
from bcc import BPF

bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
    char fname[128];
};

BPF_PERF_OUTPUT(events);

int syscall__execve(struct pt_regs *ctx, const char __user *filename) {
    struct data_t data = {};

    // Имя запускаемого файла (первый аргумент execve)
    bpf_probe_read_user_str(&data.fname, sizeof(data.fname), filename);

    data.pid = bpf_get_current_pid_tgid() >> 32;
    data.uid = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));

    events.perf_submit(ctx, &data, sizeof(data));
    return 0;
}
"""

b = BPF(text=bpf_text)
execve_fnname = b.get_syscall_fnname("execve")
b.attach_kprobe(event=execve_fnname, fn_name="syscall__execve")

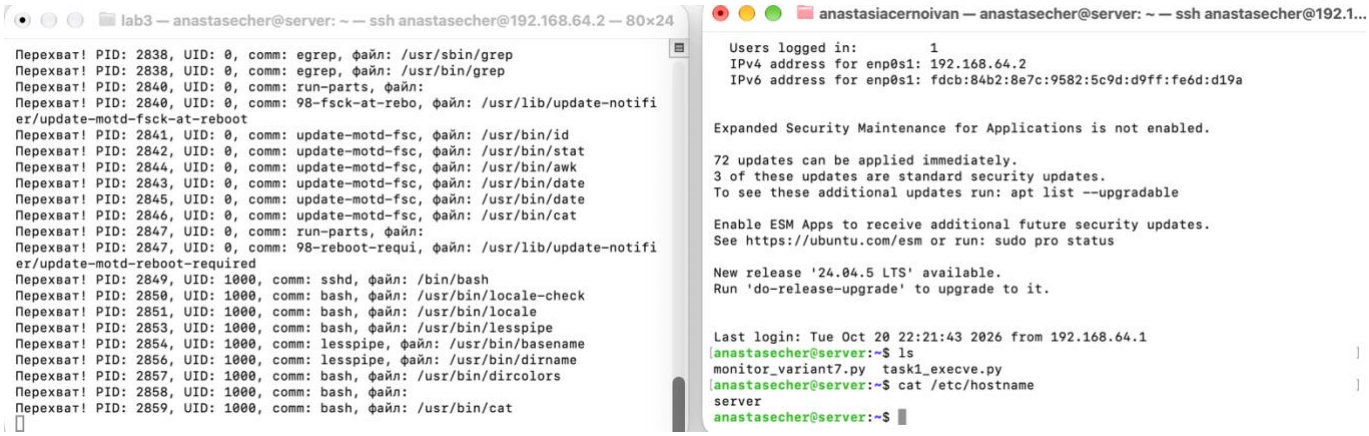
print("[*] Мониторинг запущен. Ожидание вызовов execve... (Ctrl+C для выхода)")


def print_event(cpu, data, size):
    event = b["events"].event(data)
    print(f"Перехват! PID: {event.pid}, UID: {event.uid}, "
          f"comm: {event.comm.decode('utf-8', 'replace')}, "
          f"файл: {event.fname.decode('utf-8', 'replace')}")


b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

2. Скриншот вывода терминала, демонстрирующий перехват запуска команд ls или cat, с отображением PID и имени процесса (comm).



Задание 2 (Динамический анализ аномального поведения):

1. Сформулированное условие из индивидуального варианта.

Целевой системный вызов: sys_enter_mkdirat. Условие аномалии: попытка создания скрытой директории, то есть имя создаваемой директории начинается с точки (например, .hidden, /tmp/.secret). Легитимные вызовы (mkdir normal_dir) должны отсекаться в ядре и не передаваться в пространство пользователя.

2. Листинг С-кода зонда. Обязательно выделите строками логику фильтрации, которая отсекает легитимные вызовы прямо в ядре и вызывает функцию отправки данных в кольцевой буфер только при совпадении с условием.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

#define PATH_LEN 128

struct data_t {
    u32 pid;
    u32 uid;
    u32 mode;
    char comm[TASK_COMM_LEN];
    char path[PATH_LEN];
};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_mkdirat) {
    struct data_t data = {};

    // Копируем аргумент-указатель (путь) из пользовательской памяти
    // в память ядра ОДНИМ вызовом (важно для защиты от TOCTOU).
    bpf_probe_read_user_str(&data.path, sizeof(data.path), args->pathname);

    // ---------- ФИЛЬТРАЦИЯ В ЯДРЕ (начало) ----------
    // Определяем, начинается ли с точки ПОСЛЕДНИЙ компонент пути
    // (имя создаваемой директории): ".hidden", "/tmp/.x", "a/b/.c/".
    int hidden = 0;
    char prev = '/';
```

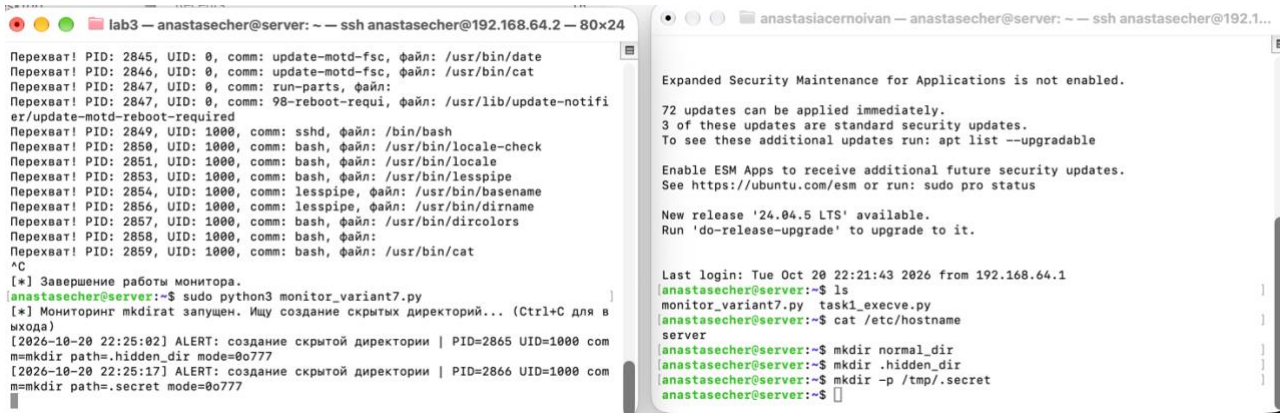
```
    #pragma unroll
    for (int i = 0; i < PATH_LEN; i++) {
        char c = data.path[i];
        if (c == 0)
            break;
        // начало нового компонента: предыдущий символ '/' (или начало строки)
        if (prev == '/' && c != '/') {
            hidden = (c == '.');
        }
        prev = c;
    }

    // Легитимные вызовы отсекаем прямо в ядре
    if (!hidden)
        return 0;
    // ---------- ФИЛЬТРАЦИЯ В ЯДРЕ (конец) ----------

    data.pid  = bpf_get_current_pid_tgid() >> 32;
    data.uid  = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    data.mode = args->mode;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));

    // Отправляем событие в user space ТОЛЬКО при аномалии
    events.perf_submit(args, &data, sizeof(data));
    return 0;
}
```

3. Скриншот двух терминалов: в первом — запуск вашего Python-монитора, во втором — генерация целевой аномалии (например, cat /etc/shadow или chmod 777 file.txt). В первом терминале должен быть зафиксирован алерт.





Задание 3 (Интеграция с пространством пользователя и аудит):

1. Фрагмент измененного Python-скрипта, реализующий запись алертов в файл security_audit.log.

```
LOG_FILE = "security_audit.log"

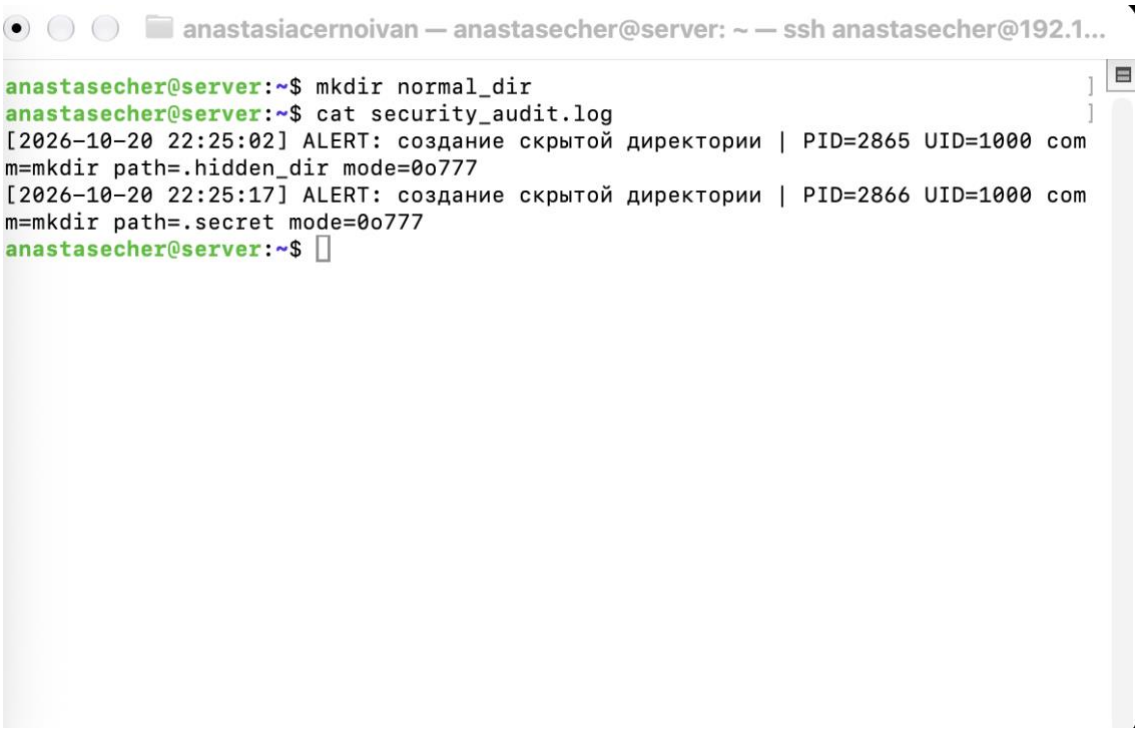
...

def print_event(cpu, data, size):
    event = b["events"].event(data)
    comm = event.comm.decode("utf-8", "replace")
    path = event.path.decode("utf-8", "replace")
    ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    msg = (f"[{ts}] ALERT: создание скрытой директории | "
           f"PID={event.pid} UID={event.uid} comm={comm} "
           f"path={path} mode={oct(event.mode)}")
    print(msg)

    # Задание 3: аудит в файл
    with open(LOG_FILE, "a", encoding="utf-8") as f:
        f.write(msg + "\n")
```

2. Текстовый вывод или скриншот содержимого файла security_audit.log с зафиксированной аномалией (должны присутствовать Timestamp, PID, UID и суть события).



Задание 4 (Проектирование профилей защиты):

1. Развернутый ответ на пункт 1: техническое обоснование различий между Seccomp-BPF (механизм ограничения доступа) и традиционными eBPF kprobes (механизм трассировки), с объяснением причин, по которым прерывание работы системного вызова из kprobe может привести к нестабильности ОС.

Seccomp-BPF и eBPF kprobes решают разные задачи. Seccomp-BPF – механизм ограничения доступа: фильтр на классическом BPF привязывается к процессу через prctl/seccomp(2), наследуется потомками и необратим (политику можно только ужесточить). Он выполняется на входе в системный вызов, до его исполнения, и возвращает вердикт (SECCOMP_RET_ALLOW, ERRNO, KILL, TRAP и др.), который ядро гарантированно применяет. Фильтр видит только номер вызова и значения аргументов в регистрах и не может разыменовывать указатели, поэтому фильтровать по пути к файлу он не умеет.

eBPF kprobe – механизм трассировки и наблюдения. Программа прикрепляется к функции ядра для всей системы и работает как наблюдатель: она может читать аргументы и память, но возвращаемое значение обработчика не влияет на ход выполнения зондируемой функции. Поэтому стандартный зонд решает задачи мониторинга и аудита, а не предотвращения.

Прерывание системного вызова из kprobe опасно для стабильности ОС. Обработчик срабатывает в произвольной точке ядра, зачастую при отключённом вытеснении, в контексте прерывания и внутри критических секций. Если насильно прервать функцию (например, подменой возвращаемого значения через bpf_override_return, который разрешён лишь для функций с пометкой ALLOW_ERROR_INJECTION), будут пропущены её завершающие действия: останутся захваченными блокировки (взаимная блокировка), не освободятся память и счётчики ссылок, нарушится согласованность структур данных, что ведёт к зависаниям и kernel panic. Кроме того, проверка в зонде и действие ядра не атомарны (возможна гонка), а имена внутренних функций ядра меняются между версиями. Отправка сигнала из eBPF тоже асинхронна: вызов может успеть выполниться. Для безопасного предотвращения предназначены Seccomp-BPF и хуки LSM (в том числе BPF-LSM), которые спроектированы так, чтобы возвращать код ошибки до выполнения действия.

2. Развернутый ответ на пункт 2: академическое описание уязвимости TOCTOU в контексте перехвата аргументов-указателей (pointers) системных вызовов в eBPF и предложения по безопасному копированию памяти (bpf_probe_read_user_str).

TOCTOU (Time-of-Check to Time-of-Use) – состояние гонки, при котором между проверкой данных и их использованием данные успевают измениться. В

нашем зонде аргумент pathname системного вызова mkdirat является указателем на буфер в пользовательской памяти. Зонд на sys_enter_mkdirat читает строку и проверяет её (Check), а ядро позже повторно копирует тот же путь из пользовательской памяти (getname) и создаёт директорию (Use). Пока процесс многопоточный, второй поток может изменить содержимое буфера между этими моментами.

Сценарий атаки: поток A вызывает mkdirat с указателем на строку «safe_dir»; зонд читает её и считает вызов легитимным; поток B успевает записать в тот же буфер «.hidden»; ядро создаёт скрытую директорию, а аудит её не фиксирует. Возможен и обход через недоступную страницу памяти: если страница выгружена, чтение из контекста зонда завершается ошибкой (зонд не может обработать page fault), и содержимое остаётся неизвестным.

Меры противодействия. Первое: копировать строку из пользовательской памяти в память ядра один раз функцией bpf_probe_read_user_str (она ограничивает длину буфера, гарантирует завершающий нуль и явно указывает, что адрес пользовательский) и принимать решение только по копии, не обращаясь к указателю повторно. Именно так сделан зонд из задания 2. Второе: проверять код возврата чтения и считать ошибку подозрительным событием. Третье (архитектурное): выносить проверку в точку, где аргументы уже скопированы ядром и недоступны для изменения из пользовательского пространства, например на внутренние функции do_mkdirat/vfs_mkdir или LSM-хук security_path_mkdir, тогда проверка и использование работают с одной и той же копией в ядре. Четвёртое: для блокировки использовать BPF-LSM, а не kprobe, а также сопоставлять событие с результатом sys_exit.

Вывод: в ходе работы освоена разработка eBPF-зондов на C с использованием фреймворка BCC: реализован перехват запуска процессов (execve) и зонд на tracepoint sys_enter_mkdirat, который фильтрует создание скрытых директорий строго в ядре и передаёт в пространство пользователя только аномальные события. Python-часть записывает алерты в журнал security_audit.log с меткой времени, PID и UID, что позволяет вести аудит. Ограничения технологии: kprobe и tracepoint пригодны для мониторинга, но не для надёжной блокировки (нестабильность при прерывании вызова, TOCTOU при чтении указателей), поэтому для предотвращения следует применять Seccomp-BPF или BPF-LSM.